Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 7

Due date: **Friday** 29 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. GPU architecture and performance characteristics [15 Points]

1.1. Peak performance [5 Points]

Using the provided utility `deviceQuery` we can retrieve the characteristics of the NVIDIA GeForce RTX 2080 Ti GPU. The output of the utility is reported in listing 1.

Listing 1: Output of the `deviceQuery` CUDA sample.

```
 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 1 CUDA Capable device(s)

Device 0: "NVIDIA␣GeForce␣RTX␣2080␣Ti"
  CUDA Driver Version / Runtime Version          13.0 / 13.0
  CUDA Capability Major/Minor version number:    7.5
  Total amount of global memory:                 10821 MBytes (11346575360
      bytes)
  (068) Multiprocessors, (064) CUDA Cores/MP:    4352 CUDA Cores
  GPU Max Clock rate:                            1545 MHz (1.54 GHz)
  Memory Clock rate:                             7000 Mhz
  Memory Bus Width:                              352-bit
  L2 Cache Size:                                 5767168 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(131072), 2D=(131072,
     65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 32768), 2048
     layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        65536 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  1024
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
```

From listing 1 we can derive that:

- The number of Streaming Multiprocessors (SMs) is $n_{\text{SMs}} = 68$
- The number of CUDA cores per SM is $n_{\text{cores per SM}} = 64$
- The GPU maximumclock frequency is $f = 1.545$ GHz
- The CUDA Compute Capability is 7.5

1.2. Memory hierarchies and bandwidth [5 Points]

From [1] we can derive the following information about the number of floating point operations per FMA.

- For single precision (FP32) FMAs $TP_{\text{FP32}} = \frac{64}{64} = 1$
- For double precision (FP64) FMAs $TP_{\text{FP64}} = \frac{64}{2} = 32$

Thus we can compute the theoretical peak performance of an NVIDIA GeForce RTX 2080 Ti for single and double precision as:

$$\begin{aligned} P_{\text{FP32}} &= n_{\text{SMs}} \cdot \frac{n_{\text{cores per SM}}}{TP_{\text{FP32}}} \cdot n_{FMA} \cdot f \\ &= 68 \cdot \frac{64}{1} \cdot 2\text{FP32} \cdot 1.545\text{GHz} = 13450,56 \text{ GFLOPS} \\ P_{\text{FP64}} &= n_{\text{SMs}} \cdot \frac{n_{\text{cores per SM}}}{TP_{\text{FP64}}} \cdot n_{FMA} \cdot f \\ &= 68 \cdot \frac{64}{32} \cdot 2\text{FP64} \cdot 1.545\text{GHz} = 420,33 \text{ GFLOPS} \end{aligned}$$

From Listing 1 we can get the following informations

- Global Memory : 10821 MiB = 10.57 GiB
- L2 Cache Size: 5.5 MiB
- L1 and Shared Memory per SM 96 KiB
- Read only memory 64 KiB
- Registers 64 Ki registers per block, each holding 4 bytes (total 256 KiB).
- Memory bus width: $W_{\text{mem}} = 352$ bits
- Memory clock frequency: $f_{\text{mem}} = 7$ GHz

So we can compute the (theoretical) peak memory bandwidth of a GPU as:

$$\begin{aligned} B_{\text{GPU}} &= \frac{W_{\text{mem}}}{8} \cdot f_{\text{mem}} \cdot 2 \\ &= \frac{352}{8} \cdot 7\text{GHz} \cdot 2 = 616 \text{ GB/s} \end{aligned}$$

1.3. Performance model: A simple roofline model [5 Points]

Compute ridge points

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{GPU}}}$$

For single precision:

$$I_{\text{ridge, FP32}} = \frac{13450.56 \text{ GFLOPS}}{616 \text{ GB/s}} = 21.83 \text{ GLOP/byte}$$

For double precision:

$$I_{\text{ridge, FP64}} = \frac{420.33 \text{ GFLOPS}}{616 \text{ GB/s}} = 0.68 \text{ GLOP/byte}$$

We can retrieve a roofline plot for the NVIDIA GeForce RTX 2080 Ti remembering that

$$P_{\text{attainable}}(I) = \min\left(P_{\text{peak}}, B_{\text{GPU}} I\right)$$

.
In figure 1 we can see the roofline plot for the NVIDIA GeForce RTX 2080 Ti for single and double precision.

Thus, when the operational intensity of a kernel is lower than I_{ridge} the kernel is memory bound, otherwise it is compute bound.

2. Introduction to CUDA [50 Points]

2.1. Implement the simple kernels [30 Points]

The CUDA implementation of the functions in the file `simple_kernels.cu` is straightforward and it is based on the standard CUDA programming model. For everty function we first computed the correct index as `int i = blockIdx.x * blockDim.x + threadIdx.x` and then we check if the index is within the bounds of the input vectors by `if ( i < N )`. If the index is valid we perform the corresponding operation on the input

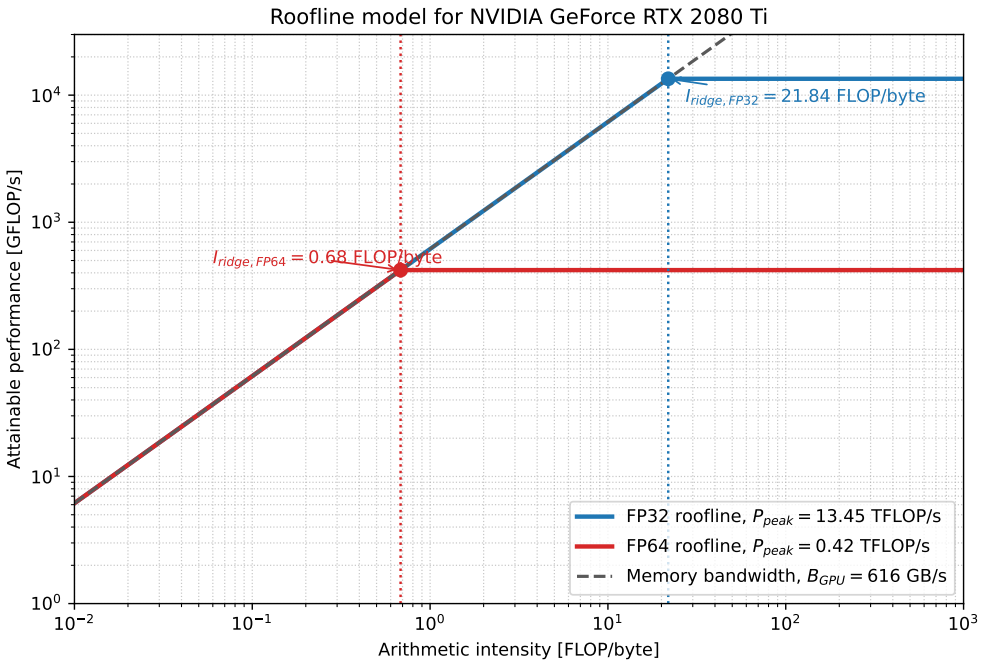


Figure 1: Roofline plot for the NVIDIA GeForce RTX 2080 Ti for single and double precision.

vectors and we store the result in the output vector.
In this case, since in the `benchmark.cu` file the number of blocks is computed as `int blocks = (SIZE + threads - 1) / threads` and the number of threads per block is set to 256, we do not need in principle to worrry about the number of threads and blocks, since the kernel will be launched with enough threads to cover all the elements of the input vectors. However, we need to check if the index is within the bounds of the input vectors to avoid out-of-bounds memory accesses.

The output of the benchmark for the double format is reported in listing 2 and for the float format in listing 3.
From 2 we can retrieve the points in table 1 and from 3 we can retrieve the points in table 2.

Listing 2: Output of the simple kernels for the double format.

```
[benchmark] axpy: warmup=5 runs=25 avg_time=0.363086 ms
Compute utilization = 46.2073 GFLOPS/s
Memory utilization = 554.487 GB/s
first = 4, last = 4, expected = 4
[benchmark] add scaled diff: warmup=5 runs=25 avg_time=0.477494 ms
Compute utilization = 52.704 GFLOPS/s
Memory utilization = 562.176 GB/s
first = 1.6, last = 1.6, expected = 1.6
[benchmark] scaled diff: warmup=5 runs=25 avg_time=0.363736 ms
Compute utilization = 46.1247 GFLOPS/s
Memory utilization = 553.496 GB/s
first = 0.6, last = 0.6, expected = 0.6
[benchmark] scale: warmup=5 runs=25 avg_time=0.247412 ms
Compute utilization = 33.9054 GFLOPS/s
Memory utilization = 542.486 GB/s
first = 0.1, last = 0.1, expected = 0.1
[benchmark] linear combination: warmup=5 runs=25 avg_time=0.363924 ms
Compute utilization = 69.1512 GFLOPS/s
Memory utilization = 553.21 GB/s
first = 1.5, last = 1.5, expected = 1.5
```

Listing 3: Output of the simple kernels for the float format.

```
[benchmark] axpy: warmup=5 runs=25 avg_time=0.182811 ms
```

Kernel	Compute Utilization (GFLOPS/s)	Arithmetic Intensity (FLOP/byte)
axpy	46.2073	$2/(3 \cdot 8)$
add scaled diff	52.704	$3/(4 \cdot 8)$
scaled diff	46.1247	$2/(3 \cdot 8)$
scale	33.9054	$1/(2 \cdot 8)$
linear combination	69.1512	$3/(3 \cdot 8)$

Table 1: Compute utilization and arithmetic intensity for the simple kernels in double precision.

Kernel	Compute Utilization (GFLOPS/s)	Arithmetic Intensity (FLOP/byte)
axpy	91.7736	$2/(3 \cdot 4)$
add scaled diff	104.988	$3/(4 \cdot 4)$
scaled diff	91.798	$2/(3 \cdot 4)$
scale	67.0205	$1/(2 \cdot 4)$
linear combination	137.747	$3/(3 \cdot 4)$

Table 2: Compute utilization and arithmetic intensity for the simple kernels in single precision.

```
Compute utilization = 91.7736 GFLOPS/s
Memory utilization = 550.642 GB/s
first = 4, last = 4, expected = 4
[benchmark] add scaled diff: warmup=5 runs=25 avg_time=0.239703 ms
Compute utilization = 104.988 GFLOPS/s
Memory utilization = 559.933 GB/s
first = 1.6, last = 1.6, expected = 1.6
[benchmark] scaled diff: warmup=5 runs=25 avg_time=0.182762 ms
Compute utilization = 91.798 GFLOPS/s
Memory utilization = 550.788 GB/s
first = 0.6, last = 0.6, expected = 0.6
[benchmark] scale: warmup=5 runs=25 avg_time=0.125165 ms
Compute utilization = 67.0205 GFLOPS/s
Memory utilization = 536.164 GB/s
first = 0.1, last = 0.1, expected = 0.1
[benchmark] linear combination: warmup=5 runs=25 avg_time=0.182696 ms
Compute utilization = 137.747 GFLOPS/s
Memory utilization = 550.989 GB/s
first = 1.5, last = 1.5, expected = 1.5
```

Figure **??** shows that all simple linear algebra kernels are located far to the left of the ridge points. Therefore, their performance is limited by global memory bandwidth rather than by the peak floating-point throughput of the GPU. This is also consistent with the measured memory bandwidths, which are close to the theoretical peak bandwidth of 616 GB/s. The single-precision kernels reach higher GFLOP/s values than the corresponding double-precision kernels because each FP32 value occupies only 4 bytes instead of 8 bytes. Hence, for the same memory bandwidth, the operational intensity and the attainable floating-point rate are approximately twice as large.

2.2. Implement the scalar product kernel [20 Points]

The first CUDA implementation of the scalar product is very simple: each thread computes some products and directly adds them to the final result using `atomicAdd`. This version is correct, but it is slow because many threads try to update the same memory location.

Listing 4: Simple CUDA kernel for the scalar product.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
int stride = blockDim.x * gridDim.x;

// Each thread works on more than one element, jumping by the total
// number of launched threads.
for (; i < N; i += stride) {
    // Simple but expensive: all threads update the same global result.
```

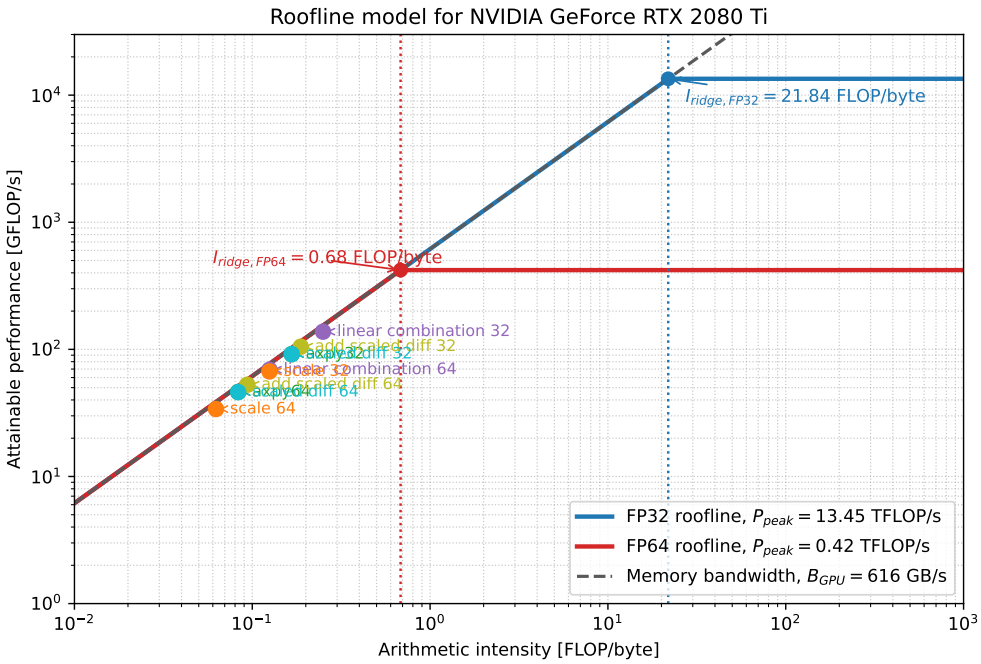


Figure 2: Roofline plot for the simple kernels in single and double precision. The points corresponding to the kernels are highlighted in red.

```
    atomicAdd(result, x[i] * y[i]);
}
```

The optimized version first accumulates a local sum inside each thread. Then the partial sums of the threads in the same block are reduced using shared memory. Only one `atomicAdd` per block is performed, which strongly reduces the contention on the global result.
An optimized version is the one in listing 5.

Listing 5: Optimized CUDA kernel for the scalar product.

```
__global__ void hpc_dot(real_t* const x, real_t* const y, real_t* result,
                        size_t const N) {

    // One partial sum per thread in the block.
    __shared__ real_t smem[256];

    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    // Each thread accumulates its own part of the dot product.
    real_t local_sum = 0.0;
    for (int idx = i; idx < N; idx += stride) {
        local_sum += x[idx] * y[idx];
    }

    // Store the thread-local result in shared memory.
    smem[tid] = local_sum;
    __syncthreads();

    // Parallel reduction inside the block.
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            smem[tid] += smem[tid + s];
        }
        __syncthreads();
```

```
27      }
28
29      // Only one thread per block updates the global result.
30      if (tid == 0) {
31          atomicAdd(result, smem[0]);
32      }
33  }
```

In table 3 we can see the compute and memory utilization for the scalar product kernel in double and single precision for both the implementations.

Version	Precision	Compute Utilization (GFLOP/s)	Arithmetic Intensity (FLOP/byte)
Simple atomic	Double	1.22579	$2/(4 \cdot 8)$
Simple atomic	Float	1.22839	$2/(4 \cdot 4)$
Block reduction	Double	71.948	$2/(2 \cdot 8)$
Block reduction	Float	140.226	$2/(2 \cdot 4)$

Table 3: Roofline points for the scalar product kernel. The block-reduction version has a higher arithmetic intensity because the main global memory traffic is due to the two input vector reads, while the global atomic update is performed only once per block.

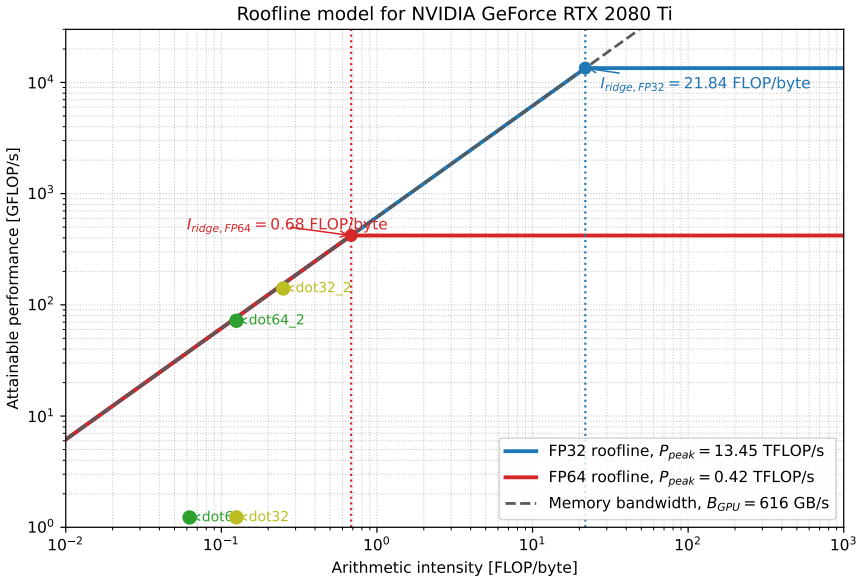


Figure 3: Roofline plot for the scalar product kernel in single and double precision. The simple atomic version is very slow because many threads contend for the same global result. The block-reduction version moves upward in the roofline plot because it performs most of the reduction in shared memory and uses only one global atomic update per block.

3. Non-linear PDE mini-app with CUDA: The Fisher equation [35 Points]

The linear algebra kernels were implemented using the same approach as for the simple kernels, thus we do not report the implementation of the kernels in this report.

The implementation of the stencil operator is reported in listing 6. Each CUDA thread computes one grid point. The kernel first ignores the external layer of points that is not updated. Then it selects the four stencil values: for an internal neighbor it reads from `s_new`, while for a boundary neighbor it uses the corresponding boundary array. This keeps the formula compact and avoids writing separate cases for all edges and corners.

Listing 6: CUDA kernel for the stencil operator.

```
__global__ void diffusion(real_t* const s_old,
                real_t* const s_new,
                real_t* f,
                real_t alpha,
                real_t beta,
                real_t* const bndE,
                real_t* const bndW,
                real_t* const bndN,
                real_t* const bndS,
                int nx) {

    int iend = nx - 1;
    int jend = nx - 1;

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Ignore threads outside the active computational domain.
    // The CUDA grid may contain more threads than needed.
    if (i < 1 || i > iend || j < 1 || j > jend) {
        return;
    }

    int idx = i + j * nx;
    real_t center = s_new[idx];

    // Use boundary values when the stencil reaches the physical boundary.
    // Otherwise, use the neighboring value from the grid.
    real_t west  = (i == 1)    ? bndW[j] : s_new[idx - 1];
    real_t east  = (i == iend) ? bndE[j] : s_new[idx + 1];
    real_t south = (j == 1)    ? bndS[i] : s_new[idx - nx];
    real_t north = (j == jend) ? bndN[i] : s_new[idx + nx];

    // Same stencil formula for internal points, edges, and corners.
    f[idx] = -(4.0 + alpha) * center
          + west + east + south + north
          + beta * center * (1.0 - center)
          + alpha * s_old[idx];
}
```

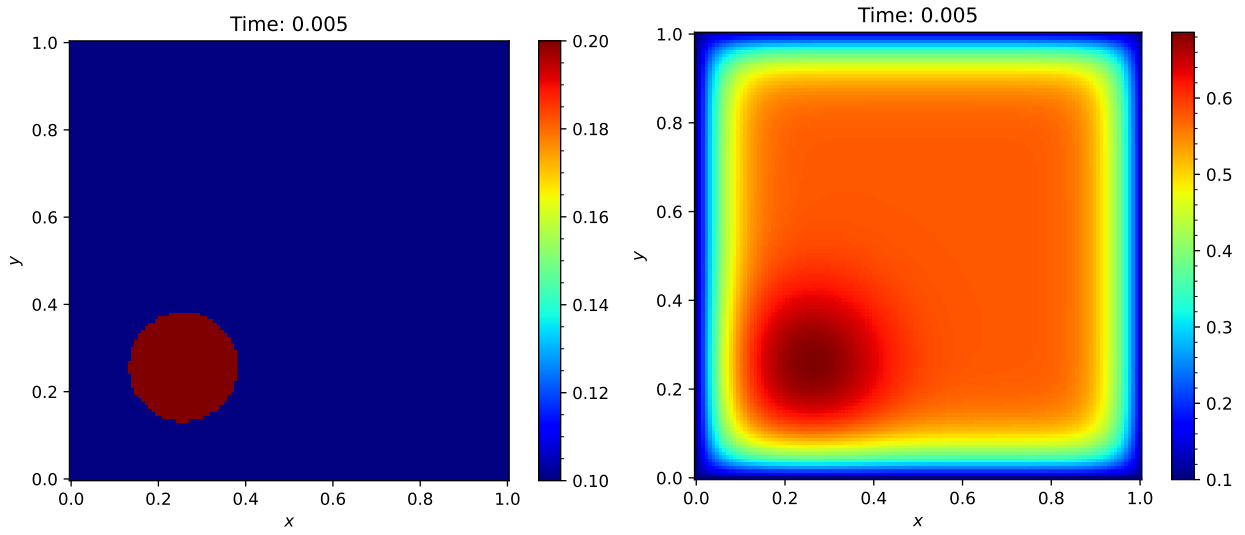
In listing 7 we can see the output of the benchmark for the Fisher equation mini-app.

The code works correctly as we can see from the plots in figure 4 where we can see the initial and final solution of the simulation. This is consistent with the expected behavior of the Fisher equation, which describes the diffusion and growth of a population over time.

Listing 7: Output of the Fisher equation mini-app.

```
================================================================
```

```
                        Welcome to mini-stencil!
version   :: CUDA
mesh      :: 128 * 128 dx = 0.00787402
time      :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
--------------------------------------------------------------------------------
simulation took 0.114354 seconds
1516 conjugate gradient iterations, at rate of 13239.6  iters/second
300 Newton iterations
--------------------------------------------------------------------------------
### 128, 100, 1514, 300, 0.114354 ###
Goodbye!
```



(a) Initial solution

(b) Final solution

Figure 4: Comparison between initial and final solution

References

[1] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. Version 13.2. 2026. URL: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/ (visited on 05/21/2026).